

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación

Restaurantes-API

Diseño General del Sistema

Gerald Montiel Quiros
Camila Thompson Sancho

I Semestre
Año lectivo 2026

Curso de Bases de Datos II
Profesor: Kenneth Obando Rodriguez

Jueves 7 de mayo de 2026

Tabla de contenidos

1. Introducción.....	2
2. Arquitectura.....	2
2.1. Lógica.....	2
2.1. Física.....	3
3. Microservicios.....	3
3. Flujo de Datos.....	4

1. Introducción

El presente documento detalla la estructura técnica y el funcionamiento de un sistema de gestión de restaurantes, diseñado bajo un modelo de microservicios contenerizados. El objetivo central de esta arquitectura es ofrecer una solución altamente escalable y flexible, capaz de gestionar de manera eficiente procesos críticos como el flujo de reservaciones, búsqueda de productos y la seguridad de los usuarios, así como la adaptación ante variaciones en motores de bases de datos.

2. Arquitectura

En esta sección se presentan las especificaciones del sistema y la arquitectura de microservicios utilizada. El diseño se aborda desde dos perspectivas complementarias, la interacción entre los servicios y cómo estos fueron implementados específicamente para este giro de negocio.

2.1. Lógica

La arquitectura lógica del sistema se basa en un modelo de capas diseñado para garantizar el desacoplamiento y la extensibilidad de las funciones. En el núcleo del backend, se implementa una capa de abstracción que permite al sistema operar correctamente sin importar la base de datos, facilitando la persistencia según las necesidades del cliente. El flujo de información se optimiza mediante una capa de caché para reducir la latencia de respuesta, mientras que la integridad y seguridad de las operaciones se centralizan a través de un servicio de autenticación. Esta estructura permite que procesos críticos como la gestión de menús, mesas y reservaciones se ejecuten de forma aislada y eficiente.

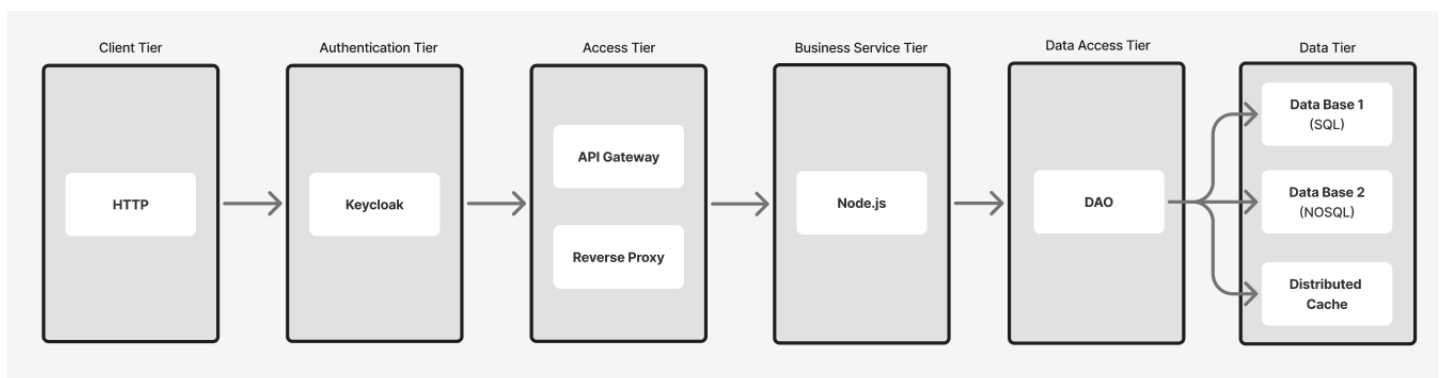


Figura 1. Diagrama de Arquitectura Lógica

2.1. Física

La implementación física se organiza mediante una infraestructura de contenedores gestionada con Docker. El sistema está diseñado para la alta disponibilidad y el rendimiento. Utiliza un balanceador Nginx que actúa como punto de entrada único, distribuyendo el tráfico hacia las instancias del servicio backend. Asimismo, la búsqueda avanzada se delega a un microservicio independiente de Elasticsearch, optimizando la indexación de productos y categorías. Finalmente, la fiabilidad del despliegue se valida mediante un pipeline de pruebas automatizadas con Jest y Supertest.

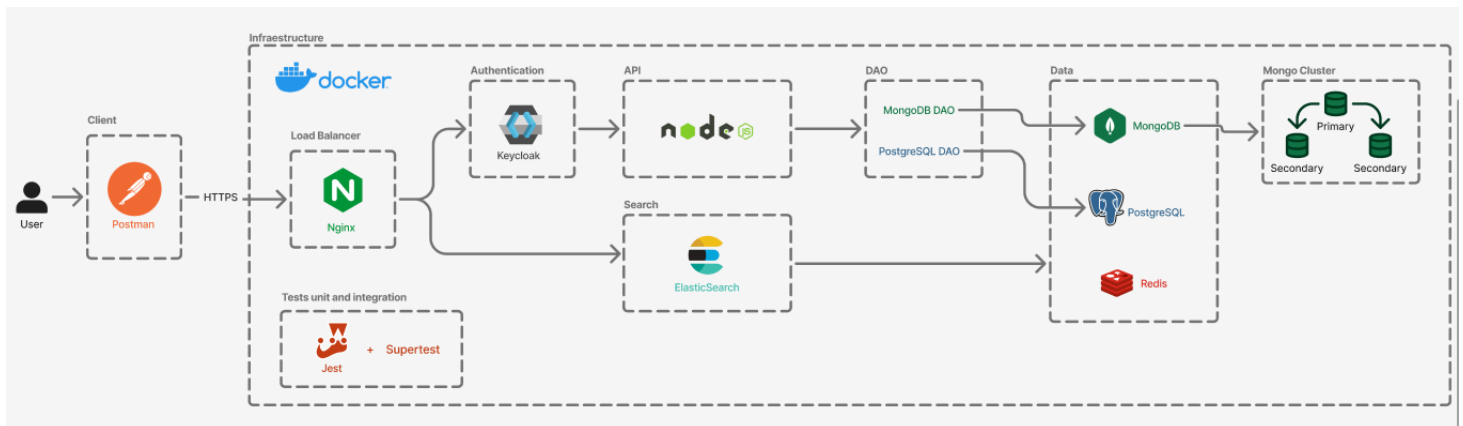


Figura 2. Diagrama de Arquitectura Física

3. Microservicios

En esta sección se detalla el propósito de cada componente (contenedor) dentro del ecosistema del sistema, especificando su rol crítico en la operación del restaurante.

- **API Gateway / Balanceador (Nginx):** Es el punto de entrada único al sistema. Se encarga de recibir todas las peticiones de los clientes, gestionar el tráfico de red y distribuirlo de manera eficiente hacia las instancias del backend para evitar cuellos de botella.

- **Servicio de Identidad (Keycloak):** Responsable de la seguridad perimetral. Administra el ciclo de vida de los usuarios, la emisión de tokens JWT y la validación de roles, asegurando que solo personal autorizado (administradores o clientes) acceda a funciones específicas.
- **Servicio Backend (Node.js):** Contiene la lógica de negocio del restaurante. Se comunica con las capas de persistencia y caché para procesar cada solicitud.
- **Motor de Búsqueda (Elasticsearch):** Microservicio especializado en la indexación y recuperación rápida de información. Su responsabilidad es permitir búsquedas para los platos ya sea por nombre o categorías.
- **Capa de Caché (Redis):** Su responsabilidad es almacenar temporalmente los datos de consulta frecuente para responder al usuario sin necesidad de consultar las bases de datos principales en cada ocasión.
- **Gestores de Persistencia (PostgreSQL / MongoDB):** Responsables del almacenamiento permanente.
 - **PostgreSQL:** Maneja datos estructurados y relaciones complejas.
 - **MongoDB:** Gestiona grandes volúmenes de datos mediante *sharding* y asegura la disponibilidad mediante su esquema de réplicas.

3. Flujo de Datos

El flujo de información en el sistema sigue una trayectoria diseñada para maximizar la velocidad de respuesta y la seguridad de los datos.

- 1. Entrada y Distribución:** Cuando un usuario realiza una acción la petición llega al balanceador Nginx. Este redirige la solicitud al contenedor de la API o del servicio de búsqueda.
- 2. Validación de Seguridad:** El backend recibe la petición y, antes de procesarla, verifica con Keycloak que el usuario tenga una sesión activa y los permisos necesarios para realizar la solicitud.
- 3. Consulta de Optimización:** El sistema primero pregunta a Redis si la información solicitada ya fue consultada recientemente.
 - *Si existe (Cache Hit):* Los datos se devuelven inmediatamente.

- *Si no existe (Cache Miss)*: Se procede a consultar la base de datos correspondiente.

4. Acceso a Datos y Búsqueda:

- Si el usuario está buscando un plato específico por texto, el flujo se dirige a Elasticsearch.
- Si es una transacción (como realizar una reserva), el flujo se dirige a PostgreSQL o MongoDB (utilizando el DAO para decidir la ruta). En MongoDB, el tráfico se distribuye entre los nodos del clúster según la configuración de *sharding*.

5. Cierre del Ciclo: Una vez obtenidos los datos, el backend procesa la respuesta, actualiza el caché en Redis si es necesario, y envía la información de vuelta al usuario a través del balanceador.